

DS210 Project Report - ChingChingYang

A. Project Overview

- a. Goal: analyze the email-Eu-core network dataset to identify the most influential nodes based on different centrality measures: Out-degree (emails sent), In-degree (emails received), Closeness centrality, Betweenness centrality
- b. Dataset:
 - i. Source: [SNAP email-Eu-core dataset](#)
 - ii. Size: 1,005 nodes, 25,571 edges

B. Data Processing

- a. Used Rust to read two text files
 - i. email-Eu-core.txt (edges between email users)
 - ii. email-Eu-core-department-labels.txt (department labels)
- b. Loaded department labels into a HashMap

C. Code Structure

- a. Modules
 - i. `graph_loader.rs`: loads graph structure and department labels from files
 - ii. `centrality.rs`: calculates out-degree, in-degree, closeness, and loads betweenness centrality
 - iii. `analysis.rs`: sorts, analyzes, and displays top nodes across centrality measures
 - iv. `main.rs`: coordinated loading, computation, and output
- b. Key Functions
 - i.

Function	Inputs	Outputs	Purpose
<code>load_graph_and_labels()</code>	File paths	(DiGraph, HashMap)	Load graph and labels
<code>calculate_centralities()</code>	Graph reference	(out_deg, in_deg, closeness, betweenness)	Compute basic centralities
<code>analyze_and_display()</code>	Graph, labels, top nodes	Printed outputs	Find overlapping top nodes and generate tables

- ii. Main workflow: load the graph and labels → calculate centralities → analyze top nodes → print results

D. Tests

- a. cargo test output:

```
running 3 tests
test analysis::tests::test_analysis_output ... ok
test centrality::tests::test centrality computation ... ok
test graph_loader::tests::test graph loading ... ok
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished
in 0.05s
```

- b.

Test	What it Checks	Why it Matters
<code>test_graph_loading</code>	Graph and labels are loaded successfully	Verifies correct file and parsing
<code>test_centrality_computation</code>	Centrality values are computed without panic	Ensures algorithm works for simple graphs
<code>test_analysis_output</code>	Analysis runs without crashing	Guarantees display functions are stable

E. Results

```

Graph loaded: 1005 nodes, 25571 edges.
Labels loaded: 1005 nodes with departments.
Top 10 nodes by out-degree (emails sent):
1. Node 160 (Dept 36): 334 emails sent
2. Node 82 (Dept 36): 227 emails sent
3. Node 121 (Dept 36): 222 emails sent
4. Node 107 (Dept 36): 204 emails sent
5. Node 86 (Dept 36): 202 emails sent
6. Node 62 (Dept 36): 190 emails sent
7. Node 13 (Dept 26): 172 emails sent
8. Node 249 (Dept 14): 160 emails sent
9. Node 183 (Dept 4): 159 emails sent
10. Node 434 (Dept 34): 157 emails sent

Top 10 nodes by in-degree (emails received):
1. Node 160 (Dept 36): 212 emails received
2. Node 62 (Dept 36): 179 emails received
3. Node 107 (Dept 36): 169 emails received
4. Node 121 (Dept 36): 157 emails received
5. Node 86 (Dept 36): 154 emails received
6. Node 434 (Dept 34): 151 emails received
7. Node 183 (Dept 4): 143 emails received
8. Node 129 (Dept 4): 139 emails received
9. Node 64 (Dept 25): 136 emails received
10. Node 128 (Dept 5): 132 emails received

Top 10 nodes by closeness centrality:
1. Node 160 (Dept 36): 0.58072
2. Node 82 (Dept 36): 0.54218
3. Node 121 (Dept 36): 0.53585
4. Node 107 (Dept 36): 0.52420
5. Node 86 (Dept 36): 0.52334
6. Node 62 (Dept 36): 0.52108
7. Node 249 (Dept 14): 0.51359
8. Node 434 (Dept 34): 0.51113
9. Node 183 (Dept 4): 0.50551
10. Node 5 (Dept 25): 0.50366

```

```

Top 10 nodes by betweenness centrality:
1. Node 160 (Dept 36): 0.07212
2. Node 86 (Dept 36): 0.03743
3. Node 5 (Dept 25): 0.02698
4. Node 121 (Dept 36): 0.02453
5. Node 62 (Dept 36): 0.02451
6. Node 107 (Dept 36): 0.02186
7. Node 64 (Dept 25): 0.01867
8. Node 82 (Dept 36): 0.01823
9. Node 377 (Dept 7): 0.01622
10. Node 129 (Dept 4): 0.01557

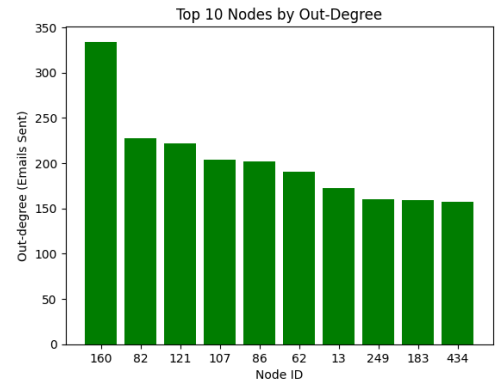
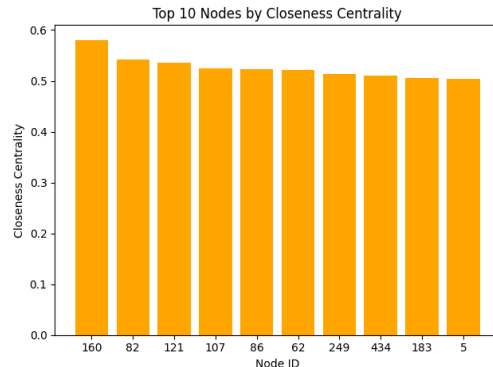
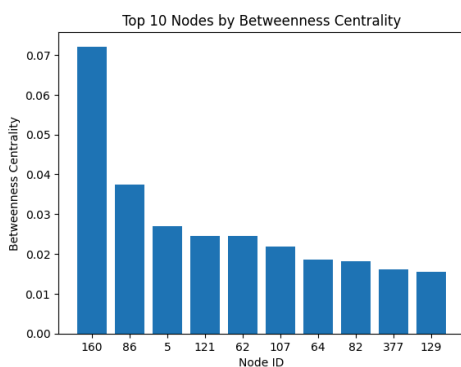
```

```

Nodes in the top 10 of ALL four centrality measures:
Node 86 (Dept 36)
Node 62 (Dept 36)
Node 107 (Dept 36)
Node 160 (Dept 36)
Node 121 (Dept 36)

```

- a.
- b. Interpretation:
 - i. Node 160 from department 36 consistently ranks highest across most measures
 - ii. Several nodes (62, 86, 107, 121) are highly influential according to multiple metrics
 - iii. Nodes with high out-degree also tend to have high closeness and betweenness scores
- c. Graphs done by Python:



F. Usage Instructions

- a. To build and run: cargo build and cargo run
- b. To run tests: cargo test
- c. No command-line arguments are needed
- d. Expected runtime: in 1 minute

G. AI-Assistance Disclosure and Other Citations

- a. I asked ChatGPT how I can compare individuals in a network based on degree, closeness, and betweenness centrality.
 - i. Degree centrality is just counting edges for each node
 - ii. Closeness centrality is calculated by the sum of shortest path lengths from each individual to all others; higher closeness = shorter distance
 - iii. Betweenness centrality is defined by counting how many times each node appears on the shortest paths between all pairs